

FITTRACK AI

Phase 1.1 — Idea Generation & Prioritization

Brainstorming & Ideation Documentation | Source: Main Project Document

1. Document Purpose

This document identifies the core functional modules and project epics described in the FitTrack AI project document and prioritizes them according to documented technical dependencies. The prioritization is dependency-driven rather than based on business preference.

2. Core Idea

FitTrack AI is a secure, scalable, AI-powered RESTful backend intended to simplify workout tracking and provide intelligent fitness recommendations. The documented system supports secure user authentication, workout management, workout searching, AI-assisted workout recommendations, and AI-generated fitness insights. It uses Node.js, Express.js, MongoDB, Mongoose, JWT authentication, bcrypt.js password hashing, and a dedicated Google Gemini AI service.

3. Core Modules Identified

ID	Core Module	Documented Scope
CM-01	Server Configuration	Node.js/Express server setup, request handling, middleware configuration, CORS and routes.
CM-02	Database Configuration	MongoDB connection and Mongoose-based schema/model configuration.
CM-03	Authentication	User registration, secure login, password hashing, JWT generation and protected APIs.
CM-04	Workout Management	Create, retrieve, update and delete workout records associated with authenticated users.
CM-05	Workout Search	Search workouts by workout name, category or workout date.
CM-06	AI Recommendation	Generate personalized workout recommendations through Google Gemini using age, goal and experience level.
CM-07	AI Fitness Insights	Analyze workout statistics and return performance analysis, improvement suggestions, motivation and progress summary.
CM-08	Middleware & Error Handling	JWT protection, request processing and centralized handling of validation, authentication, database and server errors.

4. Epic Identification

The main project document explicitly identifies Project Architecture, Project Setup and Configuration, Backend Development, Database Configuration, and Project Execution as epics. For implementation prioritization, these are mapped to the functional modules they enable.

Epic	Name	Documented Scope	Role
E-01	Project Architecture	Establish MVC/layered structure and separation of routes, controllers, services, middleware, models and AI service.	Foundation
E-02	Project Setup & Configuration	Initialize Node.js project, dependencies, environment variables and server configuration.	Foundation
E-03	Database Configuration	Configure MongoDB/Mongoose, connection, Users and Workouts schemas and relationships.	Foundation

Epic	Name	Documented Scope	Role
E-04	Backend Development	Implement authentication, workout management, AI integration, middleware and error handling.	Core Development
E-05	Project Execution & API Testing	Run and verify authentication, workout and AI APIs using Postman/ThunderClient.	Validation

5. Dependency-Based Prioritization

Priority is determined from the request flow and architecture documented in the project: the client reaches the Express server, protected routes pass through authentication middleware, controllers invoke business logic/services, Mongoose models handle database operations, and the Gemini service is used for AI functionality. Therefore, infrastructure and data/security foundations precede dependent business functions.

Order	Module / Workstream	Epic	Primary Deliverable	Dependency Rationale	Priority
P1	Project Architecture	E-01	MVC/layered structure	All application modules depend on the defined separation of routes, controllers, services, middleware and models.	High
P2	Project Setup & Configuration	E-02	Server + dependencies + environment	Required to run Express, MongoDB connectivity, authentication and Gemini configuration.	High
P3	Database Configuration	E-03	Users + Workouts schemas	Authentication and workout operations require persistent user/workout models and their documented relationship.	High
P4	Authentication & Middleware	E-04 / CM-03, CM-08	Registration, login, JWT protection	Workout and other protected APIs require a valid JWT before access.	High
P5	Workout Management	E-04 / CM-04	Workout CRUD	Core tracking functionality depends on authentication and the Workout schema.	High
P6	Workout Search	E-04 / CM-05	Search by name/category/date	Search operates on persisted workout records, so workout storage must exist first.	Medium
P7	AI Recommendation	E-04 / CM-06	Gemini recommendations	Requires the configured AI service and accepts user fitness inputs such as age, goal and experience.	Medium
P8	AI Fitness Insights	E-04 / CM-07	Gemini fitness insights	Requires the AI service and workout statistics such as total workouts, average duration and calories burned.	Medium
P9	API Testing & Execution	E-05	Authentication → Workout → AI testing	The project document specifies testing in this sequence after API implementation.	High

6. Recommended Implementation Dependency Chain

Step	Workstream	Dependency-Based Reason
1	Architecture	Define MVC/layer responsibilities and module boundaries.
2	Project Setup	Initialize Node.js/Express project and required packages/configuration.
3	Database	Connect MongoDB and define User/Workout Mongoose schemas.
4	Security Foundation	Implement bcrypt password handling, JWT service and authentication middleware.
5	Workout Core	Implement authenticated workout CRUD operations.
6	Workout Search	Add search by workout name, category and date.
7	AI Services	Integrate Gemini service and expose recommendation/insight APIs.
8	Validation	Test authentication APIs, then workout APIs, then AI APIs.

7. Scope Control

The main document mentions nutrition tracking, BMI calculation, wearable-device integration, workout reminders and fitness analytics dashboards as features that could be added later. They are therefore excluded from the current core-module prioritization and are not treated as implemented FitTrack AI functionality.

Prepared from the supplied Main Project Document. No external functionality has been introduced.